HOSTEL MANAGEMENT SYSTEM

CONSOLIDATED TEST PLAN

Quaid-i-Azam University

Software Testing Documentation

Purpose and status

A single reference file combining the performance, security, system, unit, and user acceptance testing plans for the Hostel Management System. Test cases and acceptance fields are planning material; execution results remain pending. This file does not claim that testing has been performed or that the system has passed.

Document control

Field	Details
Version / date	1.0 | 26 October 2026
Project	Hostel Management System
Institution	Quaid-i-Azam University
Testing status	Consolidated planned test cases; execution evidence pending
Included test levels	Performance, security, system, unit, and user acceptance testing

Contents

1. Performance Testing Report; 2. Security Testing Report; 3. System Testing Report; 4. Unit Testing Report; 5. User Acceptance Testing Report.

PERFORMANCE TESTING REPORT

Hostel Management System

Quaid-i-Azam University
Software Testing Documentation

Document status: Prepared test plan and test cases; execution results pending
Version: 1.0 | Date: 26 October 2026

Document Control

Field	Details
Document title	Performance Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and report documentation
Testing status	Planned test cases documented; empirical measurements not supplied
Repository destination	reports/performance-testing-report.docx on the report branch

1. Introduction

This report defines the performance-testing approach for the Hostel Management System (HMS) developed for Quaid-i-Azam University. Performance testing evaluates how the system responds to user actions and behaves when multiple users access key functions at the same time. The focus is on common student and staff workflows, database-dependent operations, and report generation.

2. Objectives

- Assess response time for common screens and user actions.
- Check whether important workflows remain usable as concurrent user activity increases.
- Observe throughput, error rate, and resource usage during representative workloads.
- Identify slow database queries, inefficient filtering, and expensive report-generation operations.
- Record reproducible measurements so future changes can be compared against a baseline.

3. Scope

Included area	Performance activity
Authentication and dashboards	Open login, authenticate a test account, and load the relevant dashboard.
Hostel applications	Submit an application and retrieve or filter application records.
Room and bed allotment	Search available beds and save an allotment using test data.
Complaint management	Submit a complaint and retrieve complaint lists/statuses.
Fee management	Retrieve fee status and record a test payment verification where supported.
Visitor management	Record visitor entry/exit and search visitor records.
Notices	Load the notice list and student-facing notices.
Reports	Generate occupancy and other available administrative reports.

Out of scope: security penetration testing, functional correctness beyond checks needed to validate test transactions, and production capacity certification. Those require separate testing and deployment information.

4. Test Environment and Preconditions

The actual implementation stack, server specifications, database engine, hosting configuration, and network conditions must be recorded before execution. These details were not provided with this report, so they are intentionally left as fields to complete rather than assumed.

Environment item	Value to record before testing
Application build / commit	_____
Application URL / deployment type	_____
Operating system and machine specifications	_____
Database engine and version	_____
Test tool and version	_____
Network conditions	_____

Test-data volume (students, applications, beds, etc.)	_____
Test date and tester	_____

Preconditions:

- Use a non-production test environment and test accounts for student and authorized staff roles.
- Prepare representative data for students, applications, hostels, rooms, beds, complaints, payments, visitors, and notices.
- Confirm that database backups or reset procedures are available before tests that create or update records.
- Ensure the test workload does not target a live university service without permission.
- Synchronize clocks and capture application, database, and load-generator logs where available.

5. Performance Metrics

Metric	Meaning	How to record
Response time	Time from request/action start until the response is usable.	Median, 90th percentile (p90), and 95th percentile (p95).
Throughput	Completed requests or transactions per second/minute.	Report by scenario and overall.
Error rate	Failed requests, timeouts, or unexpected server errors divided by total requests.	Percentage plus error categories.
Concurrent users	Number of simulated users active at the same time.	Record configured and achieved concurrency.
Resource usage	CPU, memory, database connections, and other available resource indicators.	Capture peak and average during each run.
Data integrity	Whether load caused duplicate or missing records or invalid state changes.	Validate records after the run.

6. Workload Model

Use the following as a proposed staged workload, not as a claim about the system's expected real-world capacity. Adjust the levels after discussing expected usage with the project team and confirming available test hardware.

Stage	Proposed concurrent users	Duration	Purpose
Baseline	1–5	5 minutes	Measure basic response times and validate the test setup.
Normal load	10	10 minutes	Observe common student and staff activity.
Increased load	25	10 minutes	Check response-time trends as concurrency increases.
Higher load	50	10 minutes	Explore behavior under a heavier workload, if the environment permits.
Spike test	Increase rapidly from 5 to 25	5 minutes	Observe recovery and error behavior after a sudden increase.
Soak test	10	30–60 minutes	Look for gradual degradation, resource leaks, or rising error rates.

The proposed user counts and durations are initial test settings only. They are not service-level commitments, and should be scaled to the intended deployment and available resources.

7. Performance Test Cases

ID	Scenario	Procedure / workload	Expected result	Status
PT-01	Login page load	Open the login page repeatedly with a clean test session.	Page loads successfully; record median, p90, p95 and errors.	Not run
PT-02	Login under concurrency	Run valid logins for authorized test accounts at the selected user level.	Authentication remains correct; record latency and error rate.	Not run
PT-03	Dashboard load	Load student and staff dashboards with representative data.	Correct summary data appears; record response times and failures.	Not run
PT-04	Student record search	Search for existing student records using representative search terms.	Correct records are returned; latency is recorded.	Not run
PT-05	Application filtering	Filter applications by status and other available filters.	Filtered results are correct and the response is measured.	Not run

PT-06	Submit hostel application	Submit valid applications using unique test accounts/data.	Confirmation is returned and each valid application is stored once.	Not run
PT-07	Bed availability search	Search available rooms/beds while staff accounts perform searches concurrently.	Availability results are returned; no incorrect availability is introduced.	Not run
PT-08	Bed allotment	Create allotments for distinct eligible students and beds.	Valid allotments are saved; no bed or student receives conflicting active allotments.	Not run
PT-09	Complaint submission/list	Submit test complaints and load the complaint list.	Requests complete correctly; record latency and errors.	Not run
PT-10	Fee status / verification	Retrieve fee status and verify test payments if the feature is implemented.	Records remain consistent and the response is measured.	Not run
PT-11	Visitor records	Create test visitor entries and search or update visitor records.	Visitor records are correctly saved and retrieved.	Not run
PT-12	Notice list	Load the student notice list with representative notice data.	Notices display correctly and response time is recorded.	Not run
PT-13	Occupancy report	Generate an occupancy report for a representative hostel/data range.	Report totals match database records; generation time is recorded.	Not run
PT-14	Mixed workload	Run dashboard, search, application, complaint, and report scenarios together.	System remains stable; throughput, latency, errors, and resource use are captured.	Not run
PT-15	Spike and recovery	Increase simulated users rapidly, then return to the normal workload.	Record timeouts/errors during the spike and whether normal performance recovers.	Not run
PT-16	Soak test	Run a sustained representative workload for 30–60 minutes.	No unexplained degradation, growing errors, or resource exhaustion is observed.	Not run

8. Acceptance Criteria

The project documents do not define numeric service-level targets, and no deployment benchmark was supplied. Therefore, the following are proposed criteria to confirm with the project supervisor before testing:

- Core workflows complete without unexpected failures under the agreed normal workload.
- Response-time targets are agreed and written down before the final run; report p50, p90, and p95 against those targets.
- No duplicate active bed allotments or duplicate application submissions are introduced by concurrent requests.
- Occupancy report totals and other checked outputs remain consistent with stored test data.
- Errors, timeouts, and resource consumption remain within thresholds agreed for the test environment.
- After spike testing, the application returns to the normal workload without requiring an unplanned restart.

Do not mark the system as passing solely because pages appear to load. A final pass/fail decision requires measured results and agreed numeric targets.

9. Results and Measurements

This section is intentionally unpopulated with performance numbers because no load-test output or measured execution data was supplied. Fill it after running the tests.

Scenario / ID	Users	Requests / transactions	Median (ms)	p95 (ms)	Error rate	Outcome
Baseline / PT-01	—	—	—	—	—	Pending
Normal load / PT-02–PT-13	—	—	—	—	—	Pending
Increased load / PT-14	—	—	—	—	—	Pending
Spike / PT-15	—	—	—	—	—	Pending
Soak / PT-16	—	—	—	—	—	Pending

9.1 Defect and Observation Log

ID	Scenario	Observed issue / evidence	Severity	Action / owner	Status
OBS-01	To be completed	Record measured bottlenecks, errors, or unexpected behavior.	TBD	TBD	Open
OBS-02	To be completed	Add a second row for each distinct finding.	TBD	TBD	Open

10. Recommended Execution Procedure

1. Confirm the environment, version/commit, test data volume, and approved workload.
2. Choose a load-testing tool compatible with the application's actual protocol and architecture.
3. Validate each scenario with one user before adding concurrency.
4. Run the baseline and record latency percentiles, throughput, errors, and resource use.
5. Execute the staged workload levels, keeping a separate result record for each run.
6. Check logs and database state after write-heavy scenarios; confirm there are no duplicate or inconsistent records.
7. Repeat anomalous runs to distinguish reproducible defects from temporary environment effects.
8. Attach test-tool exports, charts, timestamps, and relevant logs to the project evidence folder.
9. Update the results and defect sections, then assign pass/fail only against agreed criteria.

11. Risks and Limitations

- The deployed application and its technology stack have not been identified in this report.
- The suggested concurrent-user counts are planning values, not a verified capacity estimate.
- Network latency, machine load, database size, and test-data quality can affect results.
- Load tests against production may disrupt real users; use an authorized test environment.
- A performance run does not replace functional, integration, security, or user-acceptance testing.

12. Conclusion

This report provides a project-specific performance-testing plan for the Hostel Management System, covering its principal student and staff workflows and the measurements needed to evaluate responsiveness, concurrency, stability, and data consistency. The report is ready to be added to the project's report branch as a planned test document. Final performance conclusions must be added after execution using actual tool output and the agreed acceptance criteria.

Appendix A — Evidence Checklist

☐ Application build/commit and environment details

☐ Load-test scenario/configuration export

☐ Results for baseline, normal, increased, spike, and soak workloads

☐ Latency percentile and throughput charts

☐ Error summary and relevant application/database logs

☐ Post-run data-integrity checks

☐ Defect tickets and retest evidence, if any

Appendix B — Suggested Repository Placement

File: reports/performance-testing-report.docx

Store supporting screenshots, raw result exports, and graphs in a clearly named evidence folder if your repository structure supports it, for example: reports/evidence/performance/.

HOSTEL MANAGEMENT SYSTEM

SECURITY TESTING REPORT

Hostel Management System

Quaid-i-Azam University

Software Testing Documentation

Document status	Prepared test plan and test cases; execution results pending
Version / date	1.0 | 26 October 2026
System context	Hostel Management System (HMS), Quaid-i-Azam University
Report basis	Performance testing report supplied as format reference; system implementation details not supplied

Hostel Management System - Quaid-i-Azam University

Document Control

Field	Details
Document title	Security Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and report documentation
Testing status	Planned cases documented; no execution evidence supplied
Repository destination	reports/security-testing-report.docx on the report branch

1. Introduction

This report defines a security-testing approach for the Hostel Management System (HMS) described in the supplied performance-testing report. It focuses on protecting student and staff accounts, hostel applications, room and bed allocations, complaints, fee records, visitor information, notices, and administrative reports. The actual application, source code, deployment, and test evidence were not provided; this document is a test plan and does not claim that tests have been executed or that the system is secure.

2. Objectives

• Verify that each user can access only the functions and records allowed for their role.

• Check authentication, session handling, account recovery, and resistance to common input attacks.

• Protect sensitive student, payment, visitor, and accommodation data in transit and at rest where applicable.

• Check that security-relevant actions are logged without exposing passwords, tokens, or unnecessary personal data.

• Record reproducible evidence, findings, severity, and remediation status for each executed test.

3. Scope

Included area	Security activity
Authentication and sessions	Check login behavior, logout, session expiry, cookie attributes, and account recovery.
Role access control	Test student, warden, and administrator permissions; verify deny-by-default behavior.
Student and application records	Attempt unauthorized viewing or modification through UI and direct requests.
Room / bed allotment	Test unauthorized allotment changes, conflict manipulation, and record ownership.
Complaints and notices	Check visibility rules, staff-only actions, and input handling.
Fees and payments	Check access to fee data, verification actions, and tampering of submitted values.
Visitors	Check access restrictions and protection of visitor details.
Reports and exports	Test administrative access, filters, exports, and leakage of unrelated records.
Input handling and errors	Check injection, cross-site scripting, request forgery where relevant, and safe error responses.
Audit and configuration	Review security event logging, transport security, secrets/configuration, and dependency exposure where test access permits.

Out of scope: source-code audit, infrastructure penetration testing, production certification, and third-party service assessment unless separately authorized and provided. Testing must use a controlled non-production environment.

4. Test Environment and Preconditions

Environment item	Value to record before testing
Application build / commit	_____
Application URL / deployment type	_____
Framework, OS, and server details	_____
Database engine and version	_____
Test tools and versions	_____
Roles and test accounts	_____
Network / TLS configuration	_____
Test date and tester	_____

Preconditions

- Obtain written authorization and agree test scope, time window, accounts, and prohibited actions.
- Use an isolated test environment and synthetic records; do not test live student or financial data.
- Create least-privilege accounts for student, warden, and administrator roles, plus records owned by multiple test users.
- Back up or reset test data before state-changing tests; verify recovery steps.
- Capture request/response evidence with secrets and personal data redacted; preserve relevant application and audit logs.

5. Security Test Cases

All cases below are proposed and marked Not run. Confirm applicable roles and features against the implementation before execution. Use benign test payloads and stop if a test affects data outside the approved environment.

ID	Scenario	Procedure / expected result	Status
ST-01	Role access matrix	For each role, request permitted and restricted pages/actions. Denied actions return an appropriate denial and do not expose protected data.	Not run
ST-02	Record ownership	As one student, request another student's application, complaint, fee, or visitor record by changing identifiers. Access is denied and no data is disclosed.	Not run
ST-03	Administrative endpoints	Call staff/admin actions as a student and unauthenticated user. No protected action succeeds.	Not run
ST-04	Direct object reference	Modify record IDs in URLs and request bodies for rooms, applications, complaints, and fees. Server enforces authorization on every request.	Not run
ST-05	Login protections	Test invalid credentials, account enumeration clues, and repeated attempts within the agreed safe limit. Responses avoid revealing account existence; throttling/controls behave as specified.	Not run
ST-06	Session logout and expiry	Authenticate, log out or wait for configured expiry, then replay a prior session token. The token no longer grants access.	Not run
ST-07	Session cookie settings	Inspect cookies over HTTPS. Secure, HttpOnly, and suitable SameSite attributes are set where applicable; session identifiers rotate after authentication.	Not run
ST-08	Password recovery	Request recovery for valid and invalid accounts; inspect flow and tokens. Responses do not disclose account status; tokens are single-use and expire.	Not run
ST-09	Input injection	Submit harmless SQL/NoSQL metacharacter and query-manipulation strings in search and form fields. Inputs are safely handled; no unauthorized data or database errors appear.	Not run
ST-10	Stored/reflected XSS	Submit benign markup/script markers in complaints, notices, and profile fields. Output is encoded; no script executes in another user's browser.	Not run
ST-11	Request forgery	For state-changing actions, verify CSRF defenses or equivalent origin protections. Cross-site requests without valid protection do not change records.	Not run
ST-12	Parameter tampering	Change role, fee amount/status, student ID, allotment status, or approval fields in requests. Server-side validation rejects unauthorized changes.	Not run
ST-13	Sensitive data exposure	Inspect pages, API responses, exports, and errors for passwords, tokens, unnecessary personal data, and other users' records. Only authorized, necessary fields are returned.	Not run
ST-14	Transport security	Verify login and authenticated traffic use HTTPS in the target environment and do not downgrade or send credentials over cleartext.	Not run
ST-15	File upload / export	If uploads or exports exist, test file type/size validation and access restrictions with benign files. Files cannot execute or be retrieved by unauthorized users.	Not run
ST-16	Audit trail	Perform representative login, failed login, approval, allotment, fee verification, and role changes. Logs record actor/action/time and omit secrets.	Not run
ST-17	Error handling	Trigger invalid identifiers and malformed but benign inputs. Responses do not expose stack traces, SQL, filesystem paths, or secrets.	Not run
ST-18	Configuration review	Review test deployment for debug mode, default credentials, exposed admin/debug endpoints, and committed secrets. Record evidence and affected build.	Not run

6. Severity and Acceptance Criteria

Use the project-approved severity scheme. If none exists, record Critical / High / Medium / Low with rationale based on impact, exploitability, affected data, and required access. A security pass/fail decision requires executed tests, retained evidence, and agreed acceptance thresholds.

- No unauthorized role or user can read or modify protected records.
- No unresolved Critical or High findings remain before release, unless formally accepted by the responsible owner.
- Authentication and session controls behave as documented and protect authenticated access.
- Sensitive data is not exposed through responses, exports, logs, or error messages beyond authorized need.
- Security events needed for investigation are logged with sufficient context while excluding credentials and tokens.
- All findings have a severity, evidence, remediation owner, and retest outcome.

7. Results and Measurements

No application access, security scan output, test execution evidence, or implementation details were supplied. Therefore, this section contains no pass/fail claims. Complete it after authorized execution.

Test ID	Outcome	Evidence reference	Severity / notes
ST-01 to ST-06	Pending	_____	_____
ST-07 to ST-12	Pending	_____	_____
ST-13 to ST-18	Pending	_____	_____

7.1 Defect and Observation Log

ID	Finding / evidence	Severity	Owner	Status
SEC-01	To be completed after execution	TBD	TBD	Open
SEC-02	Add one row for each distinct finding	TBD	TBD	Open

8. Execution Notes and Report Limits

This document is a proposed security test plan based on the project name, institution, and functional areas listed in the supplied performance-testing report. The test cases may need adjustment after reviewing the actual HMS implementation and requirements. “Not run” means no test result is asserted. It is not evidence of a vulnerability or of security compliance.

9. Sign-off

Role	Name	Signature / date
Tester	_____	_____
Project supervisor	_____	_____
System owner / approver	_____	_____

HOSTEL MANAGEMENT SYSTEM

SYSTEM TESTING REPORT

Hostel Management System

Quaid-i-Azam University

Software Testing Documentation

Document status	Prepared test plan and test cases; execution results pending
Version / date	1.0 | 26 October 2026
System context	Hostel Management System (HMS), Quaid-i-Azam University
Report basis	Project workflows taken from supplied performance-testing report; implementation details not supplied

Hostel Management System - Quaid-i-Azam University

Document Control

Field	Details
Document title	System Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and report documentation
Testing status	Planned test cases documented; empirical execution results not supplied
Repository destination	reports/system-testing-report.docx on the report branch

1. Introduction

This report defines system testing for the Hostel Management System (HMS) described in the supplied project report. System testing checks complete user-facing workflows across the application and its data storage, including student and staff activities. The actual system, requirements, and execution evidence were not provided; this document is a test plan and makes no claim that tests have been run.

2. Objectives

• Verify end-to-end workflows for student, warden, and administrator roles.

• Confirm that data remains accurate as records move through application, approval, allocation, complaint, fee, and visitor workflows.

• Check validation, status transitions, search/filter behavior, and user-visible errors.

• Confirm reports and dashboards agree with underlying test records.

• Record repeatable results, defects, evidence, and retest status.

3. Scope

System area	System-level coverage
Login and dashboards	Authenticate each role and verify correct landing page, navigation, and role-specific information.
Hostel applications	Submit, review, approve/reject, search, and inspect application status.
Rooms and bed allotment	Search availability, allot eligible students, and verify room/bed occupancy updates.
Complaints	Submit complaint, view status, update through authorized workflow, and confirm history.
Fees	Retrieve fee status and record/verify test payment status where supported.
Visitors	Record visitor entry/exit and search visitor history.
Notices	Create/publish where supported and verify notice visibility for intended audience.
Reports	Generate occupancy and other available reports and reconcile output to test data.
Cross-functional checks	Verify that changes appear consistently across dashboards, searches, details, and reports.

Out of scope: unit and component tests, load/performance certification, security penetration testing, production data migration, and third-party service certification. These require separate plans and evidence.

4. Test Environment and Preconditions

Environment item	Value to record before testing
Application build / commit	_____
Application URL / deployment type	_____
Browser / device / OS	_____
Database engine and version	_____
Test accounts and roles	_____
Test-data volume and reset method	_____
Test date and tester	_____

Preconditions

• Use an approved non-production environment and synthetic test data.

• Prepare student, warden, and administrator accounts and representative records for hostels, rooms, beds, applications, complaints, fees, visitors, and notices.

• Document expected outcomes and permitted status transitions with the project team before execution.

• Confirm backup/reset steps for tests that create or update records.

• Capture screenshots or record identifiers for evidence; remove or mask personal data in shared results.

5. System Test Cases

The following cases are proposed and marked Not run. Confirm feature availability and expected rules against the implemented system before execution.

ID	Scenario	Procedure / expected result	Status
ST-01	Role login and dashboard	Log in as student, warden, and administrator. Each role reaches the appropriate dashboard and sees role-appropriate navigation/data.	Not run
ST-02	Invalid login and logout	Attempt invalid credentials, then log in and log out. Invalid login is rejected with a clear message; logout ends authenticated access.	Not run
ST-03	Student application submission	Complete a valid application with required test data. Confirmation is shown and exactly one application is stored with the expected initial status.	Not run
ST-04	Application validation	Submit missing, malformed, and boundary values. Invalid data is identified and not saved; valid values are accepted.	Not run
ST-05	Application review decision	As authorized staff, review and approve/reject a pending application. The status, decision details, and student-visible result update correctly.	Not run
ST-06	Application search/filter	Search/filter applications by supported fields and statuses. Returned records match the criteria and counts/details remain consistent.	Not run
ST-07	Bed availability search	Search available rooms/beds using supported hostel and room criteria. Results reflect current occupancy and availability rules.	Not run
ST-08	Bed allotment	Allot an available bed to an eligible student. Allocation is saved, occupancy updates, and conflicting active allotments are prevented.	Not run
ST-09	Allotment change/cancellation	Change or cancel an allotment through the supported process. Student and bed status update consistently without orphaned or duplicate allocation.	Not run
ST-10	Complaint lifecycle	Submit a complaint, view it as the student, and update/close it as authorized staff. Status and history remain consistent across views.	Not run
ST-11	Fee status workflow	View a test student fee record and perform supported verification/update. Correct status and history appear to authorized users.	Not run
ST-12	Visitor entry and exit	Record a visitor entry then exit using valid test details. The record is searchable and the visit state/time data is accurate.	Not run
ST-13	Notice publishing and visibility	Create/publish a test notice if supported. Intended student audience can view it; unpublished or restricted notices remain hidden.	Not run
ST-14	Occupancy report reconciliation	Generate the occupancy report for test hostels. Totals reconcile with active allotments and available beds under the agreed counting rules.	Not run
ST-15	Dashboard/report consistency	After representative record changes, compare dashboards, details, search results, and reports. All views reflect the same current data.	Not run
ST-16	Duplicate submission handling	Repeat a submission or action through refresh/retry where applicable. The system avoids unintended duplicate applications, payments, or allotments.	Not run
ST-17	Concurrent bed allocation	Attempt allocation of the same bed to two eligible test students in a controlled run. At most one active allocation succeeds and the other receives a clear outcome.	Not run
ST-18	Error and recovery behavior	Use unavailable/invalid test records and recover from interrupted form submission. The system shows actionable errors and does not leave inconsistent partial records.	Not run

6. Acceptance Criteria

• All agreed critical end-to-end workflows complete with expected results.

• No unresolved critical workflow defects remain; severity and release thresholds are agreed with the project owner.

• Role-specific actions and record visibility match documented requirements.

• Application statuses, allotments, complaint states, fee states, visitor records, and reports remain consistent across relevant screens.

• No duplicate active bed allocation or unintended duplicate transaction is introduced by retry or concurrent action.

• Every defect has reproducible steps, evidence, severity, owner, and retest outcome.

7. Results and Measurements

No test execution output or measured system behavior was supplied. This section is intentionally pending and must be completed after running the cases in the agreed environment. Do not assign an overall pass based only on the presence of screens.

Run / case group	Passed	Failed	Blocked / not run	Evidence / notes
Core workflows / ST-01 to ST-06	___	___	___	_____
Allocation and services / ST-07 to ST-13	___	___	___	_____
Reports and integration / ST-14 to ST-18	___	___	___	_____

7.1 Defect and Observation Log

ID	Finding / evidence	Severity	Owner	Status
SYS-01	To be completed after execution	TBD	TBD	Open
SYS-02	Add a row for each distinct defect	TBD	TBD	Open

8. Execution Notes and Report Limits

This report is a proposed system test plan based on the project workflows named in the supplied performance-testing report. It does not establish functional requirements, implementation behavior, or actual test outcomes. Review the cases with the project supervisor, update expected behavior where needed, and record execution evidence before making release decisions.

9. Sign-off

Role	Name	Signature / date
Tester	_____	_____
Project supervisor	_____	_____
System owner / approver	_____	_____

HOSTEL MANAGEMENT SYSTEM

UNIT TESTING REPORT

Hostel Management System

Quaid-i-Azam University

Software Testing Documentation

Document status	Prepared test plan and test cases; execution results pending
Version / date	1.0 | 26 October 2026
System context	Hostel Management System (HMS), Quaid-i-Azam University
Report basis	Project workflows taken from supplied performance-testing report; source code and implementation details not supplied

Hostel Management System - Quaid-i-Azam University

Document Control

Field	Details
Document title	Unit Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and report documentation
Testing status	Proposed unit test cases documented; no execution evidence supplied
Repository destination	reports/unit-testing-report.docx on the report branch

1. Introduction

This report defines a unit-testing approach for the Hostel Management System (HMS) described in the supplied project report. Unit testing verifies small, isolated functions, methods, validators, and service logic using controlled inputs and mocked dependencies. The source code, framework, and test results were not provided, so cases are expressed as behavior-focused targets to map to actual modules and are not evidence of tests already executed.

2. Objectives

• Verify core business rules independently of the user interface and live database.

• Cover valid, invalid, boundary, and exceptional inputs for critical calculations and state transitions.

• Use mocks or fixtures to isolate units from external services and prevent writes to production data.

• Record test framework, coverage, execution results, and defects so changes can be regression-tested.

3. Scope

Unit / module	Unit-level behavior to verify
Authentication helpers	Credential validation, password policy helpers, role checks, and session/token utility behavior.
Application rules	Required-field validation, eligibility rules, and allowed application status transitions.
Room / bed allocation	Availability and eligibility rules, conflict detection, capacity/occupancy calculations.
Complaint logic	Required data checks and allowed complaint status transitions.
Fee logic	Fee amount/status validation and supported verification state transitions.
Visitor logic	Visitor field validation and entry/exit state rules.
Notice logic	Required content, audience/visibility rules, and publication state transitions.
Report calculations	Occupancy totals and filtering/aggregation logic on controlled records.
Shared utilities	Date handling, identifiers, input normalization, and error mapping used by HMS modules.

Out of scope: integrated workflows, browser/UI behavior, database integration, load testing, penetration testing, and production acceptance. Those require separate test levels.

4. Test Environment and Preconditions

Environment item	Value to record before testing
Repository / branch / commit	_____
Language and framework versions	_____
Unit test runner and version	_____
Coverage tool and version	_____
Mocking / fixture strategy	_____
Test configuration	_____
Test date and tester	_____

Preconditions

• Review the codebase and map each behavior below to the actual function/class under test.

• Use isolated test configuration, deterministic fixtures, and mocks for database, email, payment, and other external services.

• Ensure tests do not connect to production services or use real student, visitor, or payment data.

• Define expected behavior and boundary rules with the project team where requirements are not documented.

• Run tests with a clean database-independent setup where possible and retain test output and coverage report.

5. Unit Test Cases

These are proposed behavior targets, not assertions about the implementation. Adapt names and expected rules after reviewing the source code and requirements. Status is Not run until execution evidence is recorded.

ID	Unit / behavior	Inputs / expected result	Status
UT-01	Required-field validator	Valid complete data passes; missing required fields fail with field-specific validation results.	Not run
UT-02	Email / identifier validation	Valid formats are accepted and malformed or unsupported values are rejected consistently.	Not run
UT-03	Password policy helper	Passwords at and beyond documented length/complexity limits produce the expected result without exposing the secret.	Not run
UT-04	Role permission helper	Allowed role/action combinations pass; disallowed and unknown roles are denied by default.	Not run
UT-05	Application eligibility rule	Eligible test profile meets configured rules; missing or disqualifying conditions return the expected reason.	Not run
UT-06	Application status transition	Only documented transitions are accepted; invalid transitions leave the original state unchanged.	Not run
UT-07	Application duplicate check	Existing duplicate criteria are detected; distinct applications remain valid according to requirements.	Not run
UT-08	Bed capacity calculation	Available/occupied counts for empty, partial, full, and boundary-capacity fixtures match expected results.	Not run
UT-09	Bed allocation conflict rule	Available bed for eligible student passes; occupied bed or conflicting active allocation is rejected.	Not run
UT-10	Allotment state transition	Valid assign/change/cancel transitions update expected state; invalid transition is rejected without side effects.	Not run
UT-11	Complaint validation and status	Required complaint fields and allowed status transitions are enforced; unsupported state changes fail.	Not run
UT-12	Fee amount / status rule	Valid nonnegative amounts and permitted fee state changes pass; invalid amounts or transitions fail.	Not run
UT-13	Visitor entry/exit rule	Entry creates expected active visit state; exit closes it; duplicate or out-of-order transitions are handled as specified.	Not run
UT-14	Notice visibility rule	Published notices are returned for intended audience; draft or restricted notice is excluded for other audiences.	Not run
UT-15	Occupancy aggregation	Controlled room/bed/allotment fixtures produce expected totals, including empty and fully occupied cases.	Not run
UT-16	Date boundary handling	Dates at configured check-in/out, deadline, or reporting boundaries are classified according to documented timezone rules.	Not run
UT-17	Error mapping helper	Known validation/domain errors map to safe expected results; unexpected exceptions do not expose internals.	Not run
UT-18	External dependency isolation	Mocked database/payment/email dependency is called with expected arguments; failures are propagated or handled as specified.	Not run

6. Coverage and Acceptance Criteria

Agree numeric coverage targets with the project team; coverage percentage alone does not demonstrate correctness. Report line/branch coverage for critical business-rule modules where the tool supports it.

- All agreed unit cases pass in the documented test environment.
- Critical business rules include valid, invalid, and boundary-condition tests.
- External dependencies are mocked or isolated; unit tests are deterministic and do not require production services.
- No unresolved critical unit-test failures remain; accepted exceptions are documented with an owner.
- Coverage report and execution output are attached to the test record, with changed modules and exclusions identified.

7. Results and Measurements

No source code, unit test output, coverage report, or test runner information was supplied. This section remains pending and does not imply a pass or fail.

Module / case group	Passed	Failed	Not run	Coverage / evidence
Validation and access / UT-01 to UT-04	___	___	___	_____
Applications and allocation / UT-05 to UT-10	___	___	___	_____
Services and shared rules / UT-11 to UT-18	___	___	___	_____

7.1 Defect and Observation Log

ID	Finding / evidence	Severity	Owner	Status
UNIT-01	To be completed after execution	TBD	TBD	Open
UNIT-02	Add a row for each distinct finding	TBD	TBD	Open

8. Execution Notes and Report Limits

This is a proposed unit test plan derived from the feature areas listed in the supplied performance-testing report. Function names, code structure, business rules, coverage targets, and actual outcomes must be confirmed against the HMS source code and project requirements before sign-off.

9. Sign-off

Role	Name	Signature / date
Tester / developer	_____	_____
Project supervisor	_____	_____
System owner / approver	_____	_____

HOSTEL MANAGEMENT SYSTEM

USER ACCEPTANCE TESTING REPORT

Hostel Management System

Quaid-i-Azam University

Software Testing Documentation

Document status	Prepared UAT plan and scenarios; execution and acceptance pending
Version / date	1.0 | 26 October 2026
System context	Hostel Management System (HMS), Quaid-i-Azam University
Report basis	Project workflows taken from supplied performance-testing report; approved requirements not supplied

Hostel Management System - Quaid-i-Azam University

Document Control

Field	Details
Document title	User Acceptance Testing Report
Project	Hostel Management System
Institution	Quaid-i-Azam University
Document version	1.0
Prepared for	Project testing and acceptance documentation
Testing status	Scenarios documented; user execution and sign-off pending
Repository destination	reports/user-acceptance-testing-report.docx on the report branch

1. Introduction

This report provides a user acceptance testing (UAT) plan for the Hostel Management System (HMS) described in the supplied project report. UAT lets representative students and hostel staff confirm that the system supports their agreed day-to-day tasks. The system, approved requirements, and user execution evidence were not provided. The scenarios below are proposed for review and do not represent completed acceptance or a release approval.

2. Objectives

• Confirm the system supports agreed student and hostel staff workflows in realistic use.

• Check that users can complete tasks and understand confirmations, statuses, and error messages.

• Identify business-rule gaps, usability issues, and missing requirements before acceptance.

• Capture user outcomes, defects, workarounds, and decisions in a traceable sign-off record.

3. Scope and Participants

Participant group	Acceptance focus
Student representatives	Apply for accommodation, check application status, view notices, submit complaints, and view personal fee information where available.
Warden / hostel staff	Review applications, manage room/bed allotments, respond to complaints, record visitors, and access operational views where authorized.
Finance staff (if applicable)	Review fee status and verify test payment records according to the approved process.
System administrator / project owner	Confirm administrative reports, access arrangements, operational readiness, and unresolved acceptance decisions.

Included: business workflows, role-specific outputs, data accuracy from a user perspective, and usability feedback. Out of scope: code-level testing, penetration testing, performance certification, and live production transactions.

4. UAT Environment and Readiness

Readiness item	Value to complete before UAT
Application build / version	_____
UAT URL / environment	_____
Approved requirements / criteria	_____
Participants and assigned roles	_____
Test accounts and synthetic data	_____
UAT dates and support contact	_____
Defect recording location	_____

Entry conditions

• Project team has identified a stable UAT build and provided access to the agreed non-production environment.

• Business owner and user representatives review and agree scenario outcomes before execution.

• Test accounts and realistic synthetic records are prepared; no live student or payment data is used.

• Known limitations, support contact, defect logging process, and reset/recovery steps are shared with participants.

• Users have received a brief orientation and can distinguish a defect from a change request or training question.

5. User Acceptance Scenarios

Participants should execute scenarios using their assigned roles, record actual results and evidence, and mark each Pass, Fail, Blocked, or Not applicable. Expected outcomes below must be confirmed against approved university procedures before the session.

ID	User scenario	Expected user outcome	Status
UAT-01	Student signs in and opens dashboard	Student reaches the correct home page and can locate the available hostel services.	Pending
UAT-02	Student submits hostel application	Required information is clear; valid application is submitted and a confirmation/reference and status are shown.	Pending
UAT-03	Student corrects incomplete application	Missing or invalid information is explained and can be corrected without losing entered data where expected.	Pending
UAT-04	Student checks application status	Student can find their own application and understand its current status and next step.	Pending
UAT-05	Staff reviews an application	Authorized staff can find a pending application, review its details, and record the approved decision.	Pending
UAT-06	Student receives decision	Student can see the updated decision/status and any relevant instruction or notification.	Pending
UAT-07	Staff finds available room/bed	Staff can search availability using the information needed for hostel operations and identify a suitable bed.	Pending
UAT-08	Staff allots bed to eligible student	Allotment is saved and user can confirm the student and bed details; conflicts are clearly prevented or reported.	Pending
UAT-09	Staff changes/cancels allotment	Authorized staff can complete the approved change process and see the resulting occupancy accurately.	Pending
UAT-10	Student submits a complaint	Student can enter complaint details and receives confirmation that it was recorded.	Pending
UAT-11	Staff updates complaint	Staff can locate and update a complaint; the resulting status is understandable to the student.	Pending
UAT-12	Student views fee status	Student can view their own fee status and understand any amount or action due, if the feature is provided.	Pending
UAT-13	Staff records visitor entry/exit	Staff can record and retrieve a visitor event using the approved process and can recognize whether a visit is active or complete.	Pending
UAT-14	User views notices	Students can find current notices intended for them and distinguish important details such as date and audience.	Pending
UAT-15	Administrator reviews occupancy report	Authorized administrator can generate the report and confirm totals against agreed test records and definitions.	Pending
UAT-16	User navigates and recovers from errors	Common tasks are understandable; validation messages explain how to continue without unintended data loss.	Pending
UAT-17	Cross-role information consistency	After an approved change, relevant student and staff views show a consistent status and current information.	Pending
UAT-18	User provides overall usability feedback	Participant records confusing steps, missing information, accessibility concerns, or process gaps for review.	Pending

6. Exit and Acceptance Criteria

• All agreed business-critical scenarios have been executed or have a documented reason for being blocked/not applicable.

• No unresolved critical acceptance defects remain; lower-severity issues have an agreed owner and disposition.

• Users and the business owner agree that key workflows and outcomes match approved requirements and operational procedures.

• Known limitations, workarounds, training needs, and deferred changes are documented and accepted by the appropriate owner.

• The authorized business owner records a clear accept, accept with conditions, or reject decision.

7. Execution Summary

UAT has not been executed. Complete this summary during the acceptance session; do not infer acceptance from this plan or from technical test results.

Summary item	Result
Execution dates / participants	_____
Scenarios passed / failed / blocked	_____
Open critical / high issues	_____
Conditions, workarounds, or deferred items	_____
Overall decision	Pending: Accept / Accept with conditions / Reject

7.1 Feedback and Defect Log

ID	User / scenario	Feedback or issue	Priority	Owner / status
UAT-OBS-01	To be completed	Record evidence and user impact	TBD	TBD
UAT-OBS-02	Add one row per issue	Record evidence and user impact	TBD	TBD

8. Approval and Sign-off

Signing confirms the recorded UAT decision for the identified build and scope. Any conditions or accepted open items should be listed in the execution summary above.

Approver role	Name	Decision	Signature / date
Student representative	_____	Accept / Conditional / Reject	_____
Hostel staff representative	_____	Accept / Conditional / Reject	_____
Business owner / project supervisor	_____	Accept / Conditional / Reject	_____

9. Report Limits

This report is a proposed UAT plan based on the functional areas described in the supplied performance-testing report. It is not a completed UAT result, an endorsement by university users, or authorization to deploy. Confirm actual workflows and acceptance criteria with the responsible stakeholders before execution.